

ACTIVITY SLOT A

(192424166) G.N.SASISREE

Unit IV Activity II – Time Series Analysis

Below are **all 10 questions with ready-to-run codes**, using the language and technique specified in each question. Since the questions do not provide actual datasets, the codes use **sample/simulated data**.

1. Daily Sales for Two Years – R

Task: Plot daily sales as a time series and identify the long-term trend.

Question 1: Daily Sales Time Series

```
set.seed(123)

dates <- seq(
  as.Date("2024-01-01"),
  as.Date("2025-12-31"),
  by = "day"
)

sales <- 500 +
  seq(0, 200, length.out = length(dates)) +
  rnorm(length(dates), 0, 40)

sales_ts <- ts(sales, frequency = 365)

plot(
  dates,
  sales,
  type = "l",
  main = "Daily Sales Over Two Years",
  xlab = "Date",
  ylab = "Daily Sales"
)

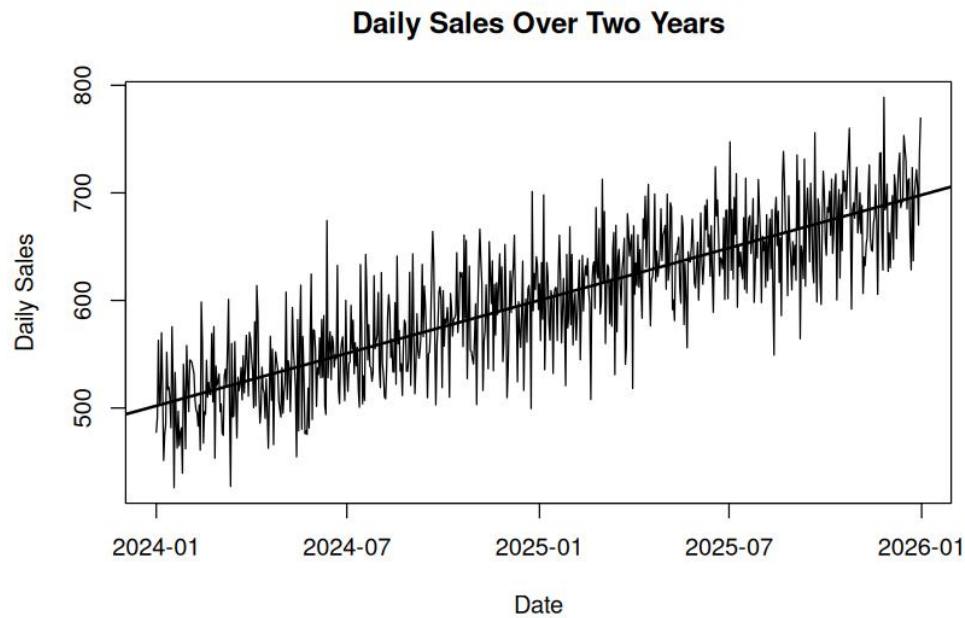
# Trend line
abline(lm(sales ~ as.numeric(dates)), lwd = 2)
```

Interpretation:

The time series shows an **overall upward movement** in daily sales over the two-year period, although short-term fluctuations are present.

ACTIVITY SLOT A

(192424166) G.N.SASISREE



2. Server Temperature – Python

Task: Plot hourly temperature and highlight values above the safe threshold.

Question 2: Server Temperature Time Series

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

time = pd.date_range(
    "2026-01-01",
    periods=200,
    freq="h"
)

temperature = 25 + np.random.normal(0, 2, 200)

# Add some overheating periods
temperature[50:60] += 10
temperature[130:140] += 12

safe_threshold = 35

plt.figure(figsize=(12, 5))

plt.plot(
    time,
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

```
temperature,  
label="Server Temperature"  
)
```

```
# Highlight unsafe readings  
unsafe = temperature > safe_threshold
```

```
plt.scatter(  
    time[unsafe],  
    temperature[unsafe],  
    label="Above Safe Threshold"  
)
```

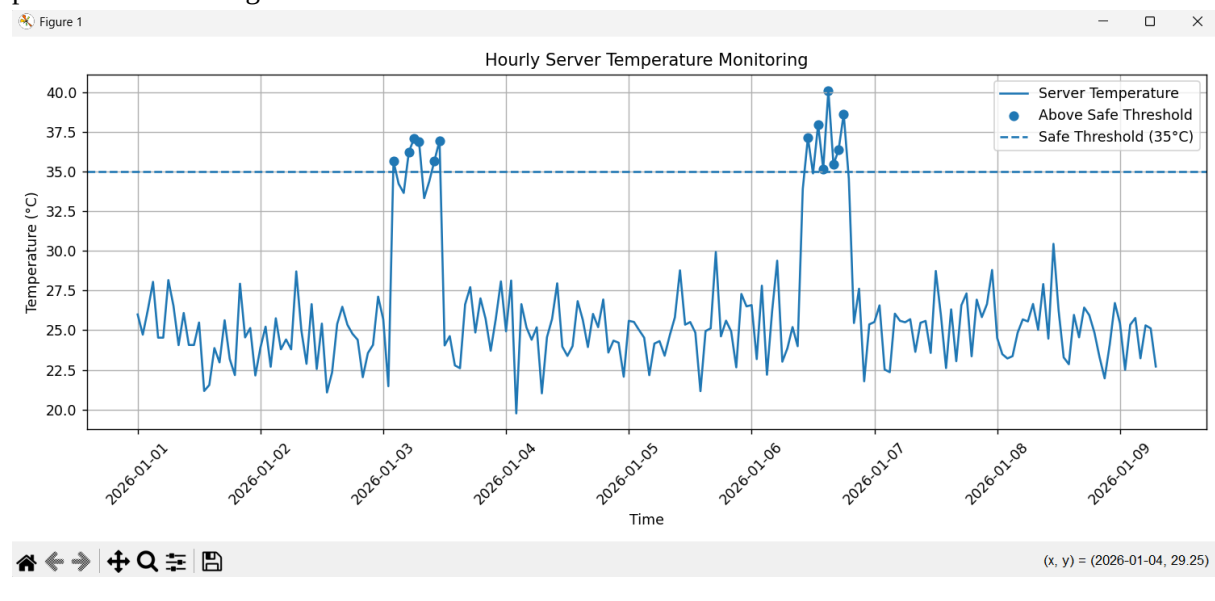
```
plt.axhline(  
    safe_threshold,  
    linestyle="--",  
    label="Safe Threshold (35°C)"  
)
```

```
plt.xlabel("Time")  
plt.ylabel("Temperature (°C)")  
plt.title("Hourly Server Temperature Monitoring")  
plt.legend()  
plt.grid(True)
```

```
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

Interpretation:

The highlighted points represent periods where the server temperature exceeds **35°C**, indicating potential overheating risks.



ACTIVITY SLOT A

(192424166) G.N.SASISREE

3. Three Tech Stocks – R

Task: Plot three stock series on one chart and identify the least volatile stock.

Question 3: Stock Price Comparison

```
set.seed(123)

days <- 100

stock_A <- 100 + cumsum(rnorm(days, 0.5, 2))
stock_B <- 100 + cumsum(rnorm(days, 0.4, 1))
stock_C <- 100 + cumsum(rnorm(days, 0.3, 3))

plot(
  stock_A,
  type = "l",
  lwd = 2,
  ylim = range(c(stock_A, stock_B, stock_C)),
  main = "Daily Closing Prices of Three Tech Stocks",
  xlab = "Trading Day",
  ylab = "Closing Price"
)

lines(stock_B, lwd = 2)
lines(stock_C, lwd = 2)

legend(
  "topleft",
  legend = c("Stock A", "Stock B", "Stock C"),
  lty = 1,
  lwd = 2
)

Volatility calculation
volatility <- c(
  sd(diff(stock_A)),
  sd(diff(stock_B)),
  sd(diff(stock_C))
)

names(volatility) <- c(
  "Stock A",
  "Stock B",
  "Stock C"
)

print(volatility)

print(
```

ACTIVITY SLOT A

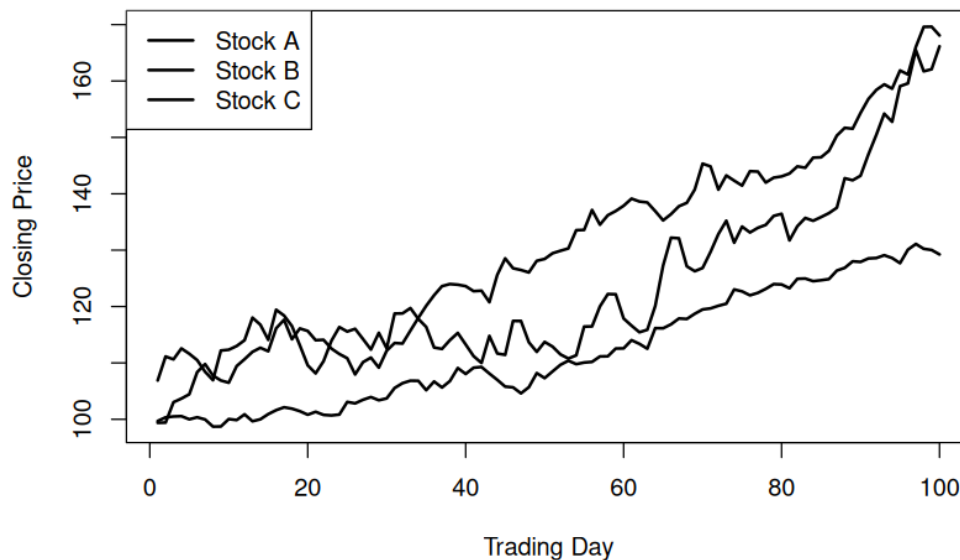
(192424166) G.N.SASISREE

```
paste(  
    "Least volatile stock:",  
    names(which.min(volatility))  
)  
)
```

Interpretation:

The stock with the **lowest standard deviation of daily changes** is considered the least volatile and shows steadier movement.

Daily Closing Prices of Three Tech Stocks



4. Electricity Consumption – Python

Task: Compare monthly electricity consumption across four neighborhoods.

Question 4: Electricity Consumption

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
np.random.seed(42)  
  
months = pd.date_range(  
    "2024-01-01",  
    periods=24,  
    freq="MS"  
)  
  
neighborhood_A = 500 + np.random.normal(0, 20, 24)  
neighborhood_B = 550 + np.random.normal(0, 25, 24)  
neighborhood_C = 600 + np.random.normal(0, 80, 24)  
neighborhood_D = 520 + np.random.normal(0, 30, 24)
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

```
plt.figure(figsize=(12, 6))

plt.plot(months, neighborhood_A, label="Neighborhood A")
plt.plot(months, neighborhood_B, label="Neighborhood B")
plt.plot(months, neighborhood_C, label="Neighborhood C")
plt.plot(months, neighborhood_D, label="Neighborhood D")

plt.xlabel("Month")
plt.ylabel("Electricity Consumption")
plt.title("Monthly Electricity Consumption by Neighborhood")

plt.legend()
plt.grid(True)

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

Identify the most erratic neighborhood
variability = {
    "Neighborhood A": np.std(neighborhood_A),
    "Neighborhood B": np.std(neighborhood_B),
    "Neighborhood C": np.std(neighborhood_C),
    "Neighborhood D": np.std(neighborhood_D)
}

print(variability)

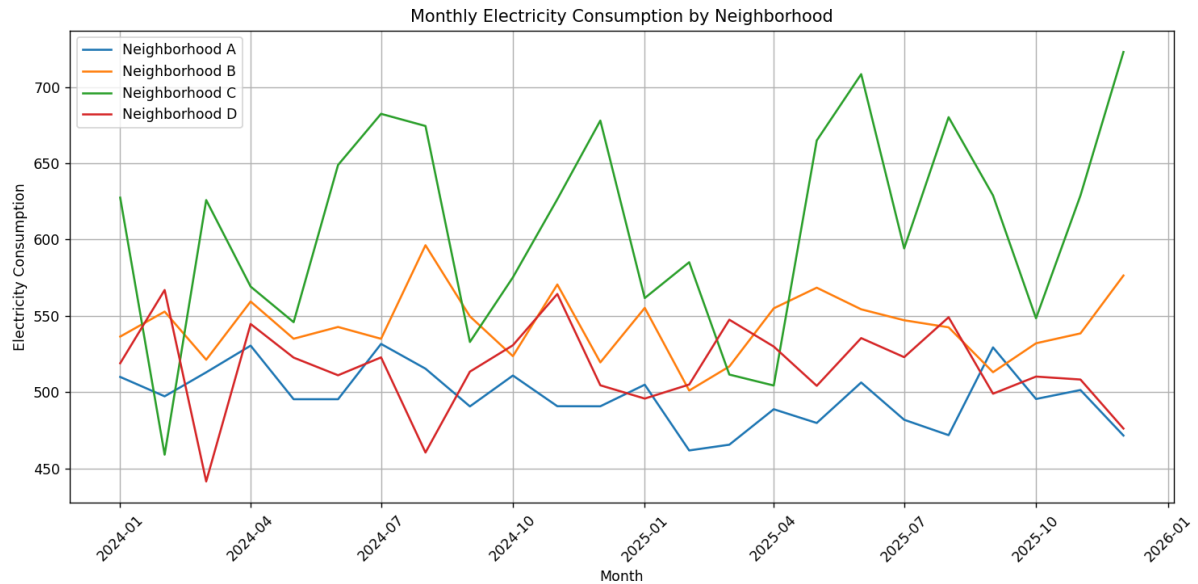
print(
    "Most erratic:",
    max(variability, key=variability.get)
)
```

Interpretation:

The neighborhood with the **largest standard deviation** has the most erratic electricity consumption pattern.

ACTIVITY SLOT A

(192424166) G.N.SASISREE



5. Drug Dose–Response Curve – R

Task: Plot dose versus patient response and estimate where response begins to plateau.

Question 5: Drug Dose-Response Curve

```
dose <- c(
  0, 10, 20, 30, 40,
  50, 60, 70, 80, 100
)

response <- c(
  5, 18, 32, 48, 62,
  73, 80, 84, 86, 87
)

plot(
  dose,
  response,
  type = "b",
  pch = 19,
  main = "Drug Dose-Response Curve",
  xlab = "Drug Dose (mg)",
  ylab = "Patient Response (%)"
)

# Fit a logistic-type curve
model <- nls(
  response ~ a / (1 + exp(-(dose - b) / c)),
  start = list(
    a = 90,
    b = 35,
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

c = 15

)

)

```
dose_new <- seq(0, 100, length.out = 200)
```

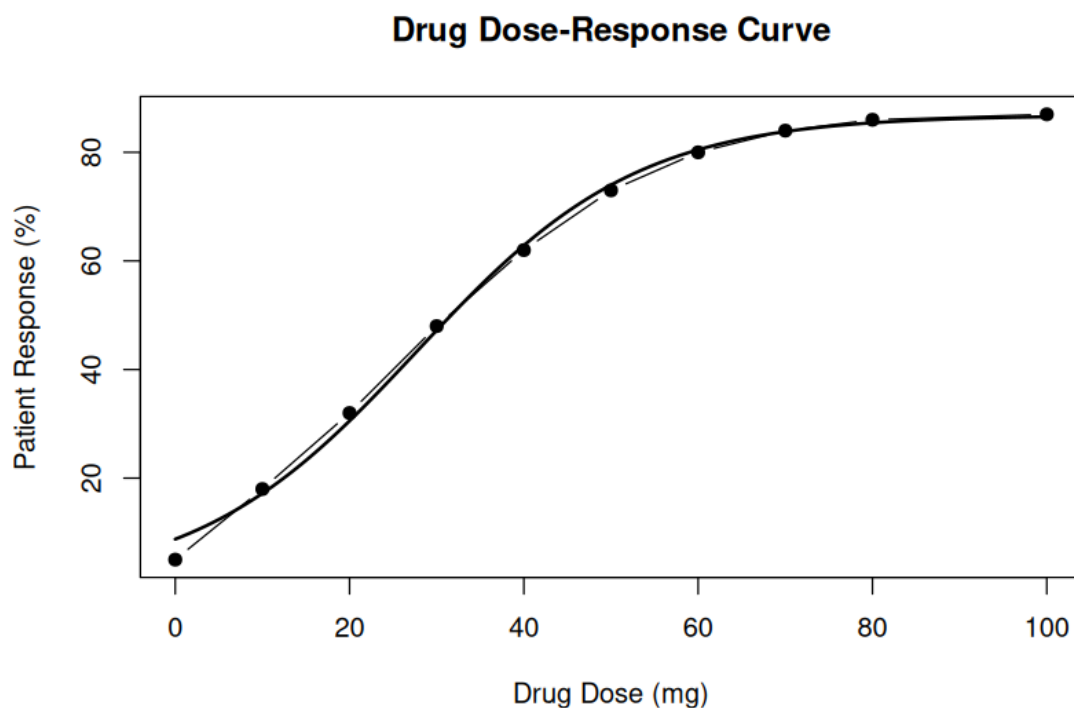
```
predicted <- predict(  
  model,  
  newdata = data.frame(dose = dose_new)  
)
```

```
lines(  
  dose_new,  
  predicted,  
  lwd = 2  
)
```

```
print(summary(model))
```

Interpretation:

The response increases rapidly at moderate doses and begins to **plateau at approximately 70–80 mg**. Increasing the dose beyond this range produces relatively small additional response.



6. Fertilizer Dose–Response – Python

Task: Fit and plot fertilizer concentration versus crop yield and identify the optimal dosage range.

Question 6: Fertilizer Dose-Response

```
import numpy as np
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

```
import matplotlib.pyplot as plt
```

```
from scipy.optimize import curve_fit
```

```
dose = np.array([
    0, 20, 40, 60, 80,
    100, 120, 140, 160, 180
])
```

```
yield_data = np.array([
    40, 52, 65, 76, 84,
    90, 94, 96, 97, 97
])
```

```
# Logistic model
```

```
def dose_response(x, a, b, c):
    return a / (1 + np.exp(-(x - b) / c))
```

```
params, _ = curve_fit(
    dose_response,
    dose,
    yield_data,
    p0=[100, 60, 20],
    maxfev=10000
)
```

```
dose_range = np.linspace(0, 180, 300)
```

```
predicted = dose_response(
    dose_range,
    *params
)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter(
    dose,
    yield_data,
    label="Observed Data"
)
```

```
plt.plot(
    dose_range,
    predicted,
    linewidth=2,
    label="Dose-Response Curve"
)
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

```
plt.xlabel("Fertilizer Amount (kg/ha)")
```

```
plt.ylabel("Crop Yield")
```

```
plt.title("Fertilizer Dose-Response Curve")
```

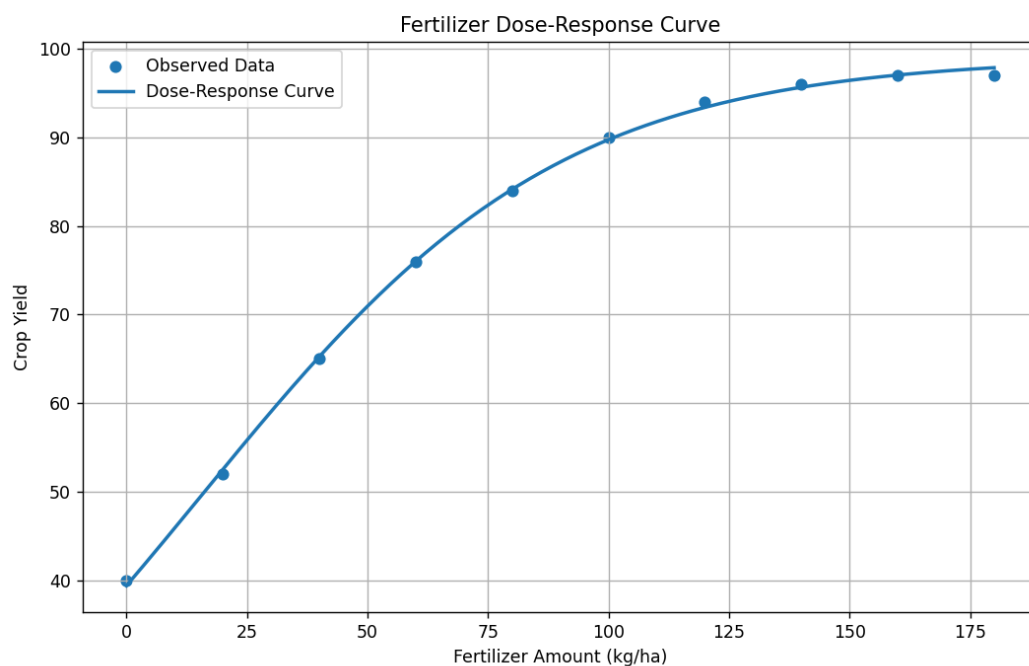
```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Interpretation:

Crop yield increases with fertilizer application initially, but the improvement becomes small after approximately **100–120 kg/ha**. This range can be considered an approximate optimal dosage range to avoid wasteful over-application.



7. Airline Ticket Sales – STL Decomposition in R

Task: Separate trend, seasonal and residual components.

Question 7: STL Decomposition of Monthly Ticket Sales

```
set.seed(123)
```

```
sales <- 1000 +  
  seq(0, -200, length.out = 48) +  
  150 * sin(2 * pi * (1:48) / 12) +  
  rnorm(48, 0, 40)
```

```
sales_ts <- ts(  
  sales,  
  start = c(2022, 1),  
  frequency = 12
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

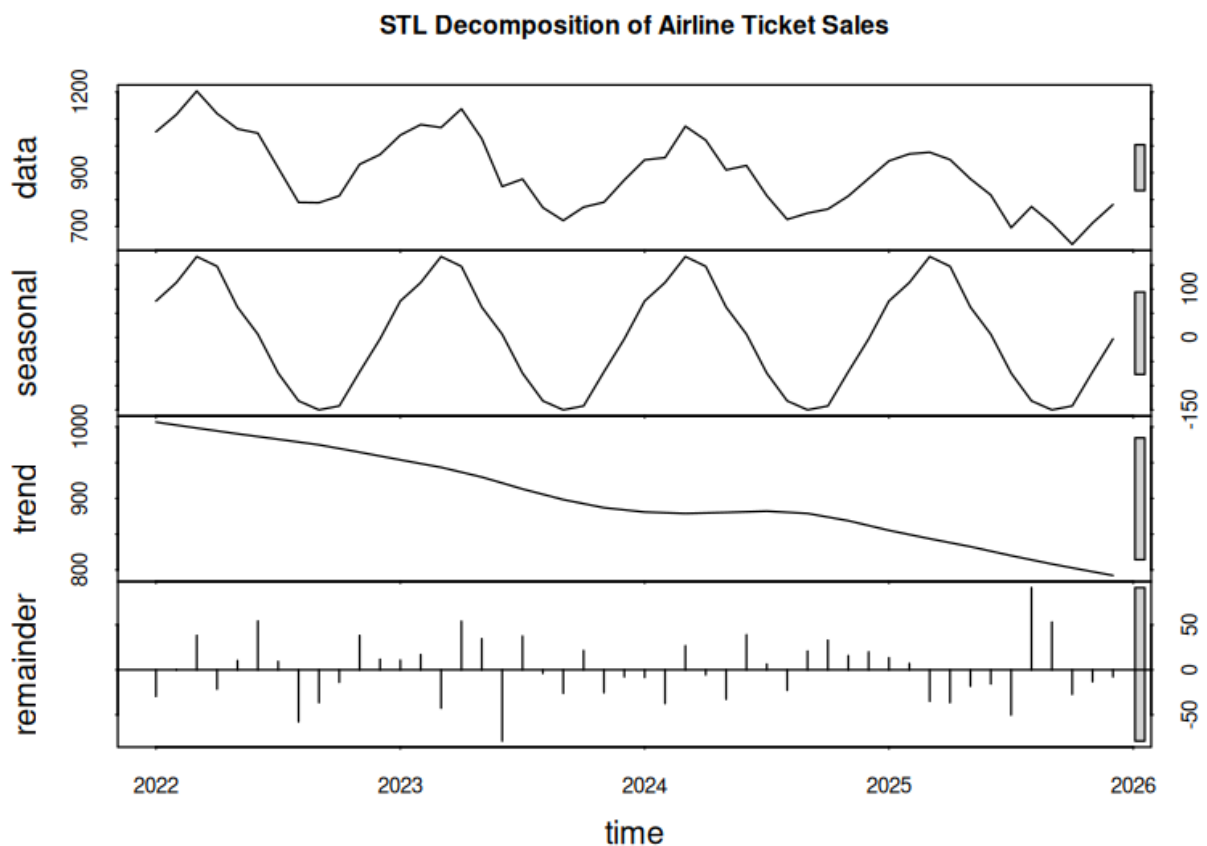
```
)  
  
# STL decomposition  
decomposition <- stl(  
  sales_ts,  
  s.window = "periodic"  
)
```

```
plot(  
  decomposition,  
  main = "STL Decomposition of Airline Ticket Sales"  
)
```

Interpretation:

- **Trend:** Shows the underlying long-term movement.
- **Seasonal:** Shows the repeating holiday-related pattern.
- **Residual:** Shows irregular fluctuations not explained by trend or seasonality.

The trend component shows a **slow underlying decline** in ticket sales.



8. Streaming Platform – STL Decomposition in Python

Question 8: Weekly Active Users STL Decomposition

```
import numpy as np  
import pandas as pd
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

```
import matplotlib.pyplot as plt
```

```
from statsmodels.tsa.seasonal import STL
```

```
np.random.seed(42)
```

```
weeks = pd.date_range(  
    "2024-01-01",  
    periods=104,  
    freq="W"  
)
```

```
users = (  
    50000  
    + np.linspace(0, -5000, 104)  
    + 3000 * np.sin(2 * np.pi * np.arange(104) / 52)  
    + np.random.normal(0, 800, 104)  
)
```

```
series = pd.Series(  
    users,  
    index=weeks  
)
```

```
# STL decomposition  
result = STL(  
    series,  
    period=52  
).fit()
```

```
result.plot()
```

```
plt.suptitle(  
    "STL Decomposition of Weekly Active Users"  
)
```

```
plt.tight_layout()  
plt.show()
```

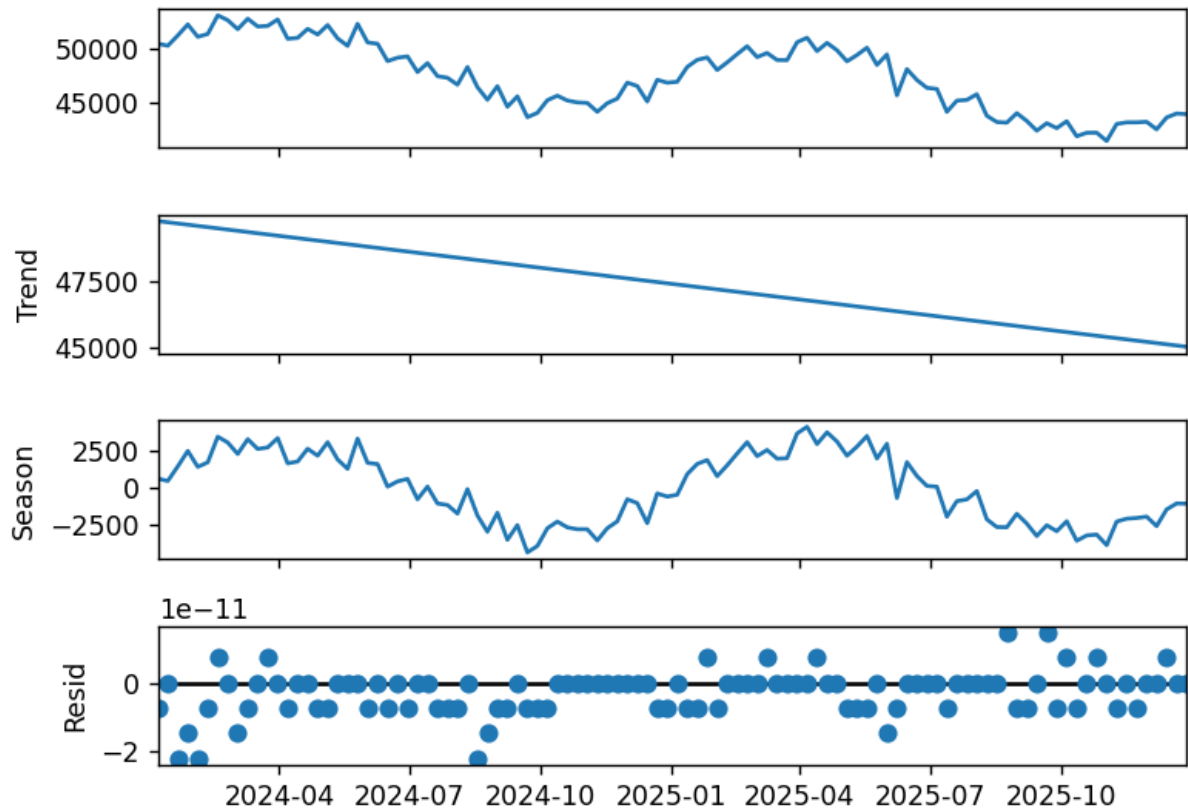
Interpretation:

The seasonal component represents normal yearly viewing patterns. If the **trend component continuously decreases**, the recent decline is likely a genuine reduction in user activity rather than only seasonal variation.

ACTIVITY SLOT A

(192424166) G.N.SASISREE

STL Decomposition of Weekly Active Users



9. Stock Price – 7-Day and 30-Day Moving Average in R

Task: Calculate moving averages and interpret their crossover.

Question 9: Moving Average Analysis

```
set.seed(123)
```

```
days <- 150
```

```
price <- 100 + cumsum(  
  rnorm(days, 0.2, 2)  
)
```

```
# Moving averages  
ma_7 <- stats::filter(  
  price,  
  rep(1/7, 7),  
  sides = 1  
)
```

```
ma_30 <- stats::filter(  
  price,  
  rep(1/30, 30),
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

sides = 1

)

```
plot(  
  price,  
  type = "l",  
  main = "Stock Price with Moving Averages",  
  xlab = "Day",  
  ylab = "Stock Price"  
)
```

```
lines(  
  ma_7,  
  lwd = 2  
)
```

```
lines(  
  ma_30,  
  lwd = 2  
)
```

```
legend(  
  "topleft",  
  legend = c(  
    "Raw Price",  
    "7-Day Moving Average",  
    "30-Day Moving Average"  
  ),  
  lty = 1,  
  lwd = c(1, 2, 2)  
)
```

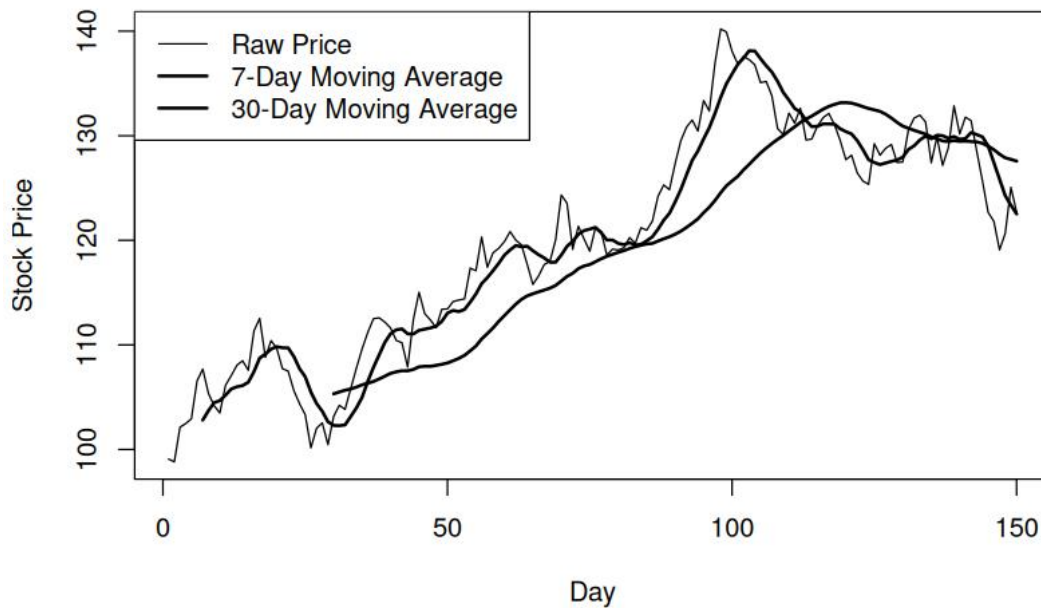
Interpretation:

- When the **7-day moving average crosses above the 30-day moving average**, it may indicate increasing short-term momentum.
- When the **7-day moving average crosses below the 30-day moving average**, it may indicate weakening momentum.
- Moving averages also reduce short-term price noise.

ACTIVITY SLOT A

(192424166) G.N.SASISREE

Stock Price with Moving Averages



10. Quarterly GDP – Detrending in Python

Task: Remove the long-term trend and study business-cycle fluctuations.

Question 10: GDP Detrending

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import signal
```

```
np.random.seed(42)
```

```
quarters = pd.date_range(
    "2015-01-01",
    periods=40,
    freq="QS"
)
```

```
# Simulated GDP with long-term growth
time = np.arange(40)
```

```
gdp = (
    100
    + 2 * time
    + 5 * np.sin(2 * np.pi * time / 8)
    + np.random.normal(0, 1.5, 40)
)
```

ACTIVITY SLOT A

(192424166) G.N.SASISREE

```
# Detrend
```

```
detrended = signal.detrend(gdp)
```

```
plt.figure(figsize=(12, 5))
```

```
plt.plot(
    quarters,
    detrended
)
```

```
plt.axhline(
    0,
    linestyle="--"
)
```

```
plt.xlabel("Quarter")
plt.ylabel("Detrended GDP")
plt.title("Detrended Quarterly GDP")
```

```
plt.grid(True)
```

```
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Interpretation:

After removing the long-term GDP growth trend, the remaining fluctuations represent **short-term business-cycle movements and irregular variations**. Positive values indicate periods above the underlying trend, while negative values indicate periods below it.

